

Blocco 12 – Capstone architettura DBMS e dashboard

Stack containerizzato con PostgreSQL, collector e Metabase

Materiali

- Durata: 90 minuti.
- Slide: `slides/blocks/block12_capstone_query_design/block12_capstone_query_design.pdf`
- Ambiente laboratorio: container Docker PostgreSQL `rdsq1-postgres`
- Schema PostgreSQL: `../..../sql/01_schema_seed_postgres.sql`
- Script soluzione: `solution.sql`
- Script unico schema + soluzioni: `../..../sql/00_schema_and_all_solutions_postgres.sql`

1. Problema in forma informale

Bisogna consegnare un piccolo sistema dati replicabile: container PostgreSQL, collector simulato, schema ticketing, query SQL e dashboard Metabase.

2. Specifica corretta del problema

- input: `docker-compose.ticketing.yml`, script SQL ticketing e Metabase;
- output: stack avviato, database popolato, dashboard con query SQL, controlli;
- vincolo: la dashboard deve usare PostgreSQL come sorgente;
- vincolo: le query devono essere versionate nei file `.sql`;
- vincolo: la presentazione deve spiegare architettura, metriche e limiti.

3. Avvio

```
docker compose -f docker-compose.ticketing.yml up -d
docker compose -f docker-compose.ticketing.yml ps
docker logs -f rdsq1-ticket-collector
```

4. Query da copiare in Metabase

```
SELECT count(*) AS total_tickets,
       count(*) FILTER (WHERE closed_at IS NULL) AS open_tickets,
       count(*) FILTER (WHERE closed_at IS NOT NULL) AS closed_tickets,
       count(*) FILTER (WHERE sla_breached) AS sla_breached_tickets
FROM ticketing.dashboard_ticket_base;

SELECT day, opened_tickets, resolved_tickets, backlog_delta
FROM ticketing.dashboard_daily_flow
ORDER BY day;
```

5. Criteri di verifica

- i tre container sono avviati;
- Metabase si collega al database **training**;
- il collector aggiunge nuove righe;
- la dashboard contiene KPI, trend, ranking e dettaglio;
- la soluzione dichiara limiti e trade-off.